

University Database Schema

- **classroom**(*building*, room number, capacity)
- **department**(*dept name*, building, budget)
- **course**(*course id*, title, dept name, credits)
- **instructor**(*ID*, name, dept name, salary)
- **section**(*course id*, sec id, semester, year, building, room number, time slot id)
- **teaches**(*ID*, course id, sec id, semester, year)
- **student**(*ID*, name, dept name, tot cred)
- **takes**(*ID*, course id, sec id, semester, year, grade)
- **advisor**(*s ID*, *i ID*)
- **time slot**(*time slot id*, day, start time, end time)
- **prereq**(*course id*, prereq id)

Employee Database Schema

- **employee**(employee_name, street, city)
- **works**(employee_name, company_name, salary)
- **company**(company_name, city)
- **manages**(employee_name, manager_name)

SQL Queries and Questions

Question 1

Find the names of all instructors.

```
SELECT name
FROM instructor;
```

```
SELECT DISTINCT name
FROM instructor;
```

Question 2

Find the names of all departments that have instructors, and remove duplicates.

```
-- Remove duplicates (default behavior with DISTINCT)
SELECT DISTINCT dept_name
FROM instructor;
```

```
-- Keep duplicates explicitly using ALL
SELECT ALL dept_name
FROM instructor;
```

Question 3

Display the ID, name, department name, and monthly salary of all instructors. Assume salary is stored as yearly salary.

```
SELECT ID, name, dept_name, salary/12 AS monthly_salary
FROM instructor;
```

Question 4

Find the names of all instructors in the Computer Science department with a salary greater than 80,000.

```
SELECT name
FROM instructor
WHERE dept_name = 'Comp. Sci.' AND salary > 80000;
```

Question 5

Find the Cartesian product of the tables `instructor` and `teaches`.

```
SELECT *
FROM instructor, teaches;
```

```
-- Using explicit CROSS JOIN
SELECT *
FROM instructor CROSS JOIN teaches;
```

Question 6

For all instructors who have taught some course, find their names and the course ID of the courses they taught.

```
-- Using implicit join
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID;
```

```
-- Using explicit JOIN syntax
SELECT name, course_id
FROM instructor JOIN teaches USING (ID);
```

```
SELECT name, course_id
FROM instructor JOIN teaches ON instructor.ID = teaches.ID;
```

```
SELECT name, course_id
FROM instructor NATURAL JOIN teaches;
```

Question 7

Find the course ID, semester, year, and title of each course offered by the Computer Science department.

```
-- Using implicit join
SELECT section.course_id, semester, year, title
FROM section, course
WHERE section.course_id = course.course_id
      AND dept_name = 'Comp. Sci.';
```

```
-- Using explicit JOIN
SELECT section.course_id, semester, year, title
FROM section JOIN course USING (course_id)
WHERE dept_name = 'Comp. Sci.';
```

```
SELECT section.course_id, semester, year, title
FROM section
JOIN course ON section.course_id = course.course_id
WHERE dept_name = 'Comp. Sci.';
```

Question 8

List the names of instructors along with the titles of the courses that they teach.

```
-- Incorrect version (may mistakenly enforce dept_name match)
SELECT name, title
FROM instructor NATURAL JOIN teaches NATURAL JOIN course;
```

```
-- Correct version using implicit join
SELECT name, title
FROM instructor NATURAL JOIN teaches, course
WHERE teaches.course_id = course.course_id;
```

```
-- Correct version using explicit JOIN
SELECT name, title
FROM (instructor NATURAL JOIN teaches)
JOIN course USING (course_id);
```

Question 9

Find the names of all instructors who have a higher salary than some instructor in the Computer Science department.

```
-- Using self-join with aliases
SELECT DISTINCT T.name
FROM instructor AS T, instructor AS S
WHERE T.salary > S.salary
      AND S.dept_name = 'Comp. Sci.';
```

```
-- Using JOIN syntax
SELECT DISTINCT T.name
FROM instructor AS T
JOIN instructor AS S
  ON T.salary > S.salary
WHERE S.dept_name = 'Comp. Sci.';
```

```
-- Using a subquery
SELECT name
FROM instructor
WHERE salary > ANY (
  SELECT salary
  FROM instructor
  WHERE dept_name = 'Comp. Sci.'
);
```

Question 10

Find the names of all instructors whose name includes the substring “dar”.

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%';
```

Question 11

List in alphabetical order the names of all instructors.

```
SELECT DISTINCT name
FROM instructor
ORDER BY name;
```

Question 12

Find the names of all instructors with salary between \$90,000 and \$100,000 (inclusive).

```
SELECT name
FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```

```
-- Equivalent without BETWEEN
SELECT name
FROM instructor
WHERE salary >= 90000 AND salary <= 100000;
```

Question 13

- Find courses that ran in Fall 2009 or in Spring 2010.
- Find courses that ran in Fall 2009 but not in Spring 2010.
- Find courses that ran in Fall 2009 and in Spring 2010.

```
-- (a) Fall 2009 or Spring 2010
(SELECT course_id
 FROM section
 WHERE sem = 'Fall' AND year = 2009)
UNION
(SELECT course_id
 FROM section
 WHERE sem = 'Spring' AND year = 2010);
```

```
-- (b) Fall 2009 but not Spring 2010
(SELECT course_id
 FROM section
 WHERE sem = 'Fall' AND year = 2009)
EXCEPT
(SELECT course_id
 FROM section
 WHERE sem = 'Spring' AND year = 2010);
```

```
-- (c) Fall 2009 and Spring 2010
(SELECT course_id
 FROM section
 WHERE sem = 'Fall' AND year = 2009)
INTERSECT
(SELECT course_id
 FROM section
 WHERE sem = 'Spring' AND year = 2010);
```

Question 14

Find all instructors whose salary is NULL.

```
SELECT name
FROM instructor
WHERE salary IS NULL;
```

Question 15

Find the average salary of instructors in the Computer Science department.

```
SELECT AVG(salary)
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

Question 16

Find the total number of instructors who teach a course in the Spring 2010 semester.

```
SELECT COUNT(DISTINCT ID)
FROM teaches
WHERE semester = 'Spring' AND year = 2010;
```

Question 17

Find the number of tuples in the `course` relation.

```
SELECT COUNT(*)
FROM course;
```

Question 18

Find the average salary of instructors in each department.

```
SELECT dept_name, AVG(salary) AS avg_salary_dept_wise
FROM instructor
GROUP BY dept_name;
```

Question 19

Find the number of instructors in each department who teach a course in the Spring 2010 semester.

```
SELECT dept_name, COUNT(DISTINCT ID) AS instr_count
FROM instructor NATURAL JOIN teaches
WHERE semester = 'Spring' AND year = 2010
GROUP BY dept_name;
```

Question 20

Find the names and average salaries of all departments whose average salary is greater than 42,000.

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

Question 21

For each course section offered in 2009, find the average total credits (`tot_cred`) of all students enrolled in the section, if the section had at least 2 students.

```
SELECT course_id, semester, year, sec_id, AVG(tot_cred) AS avg_tot_cred
FROM takes NATURAL JOIN student
WHERE year = 2009
GROUP BY course_id, semester, year, sec_id
HAVING COUNT(ID) >= 2;
```

Question 22

Find courses offered in Fall 2009 and in Spring 2010.

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
AND course_id IN (
    SELECT course_id
    FROM section
    WHERE semester = 'Spring' AND year = 2010
);
```

```
SELECT course_id
FROM section AS S
WHERE semester = 'Fall' AND year = 2009
AND EXISTS (
    SELECT *
    FROM section AS T
    WHERE semester = 'Spring' AND year = 2010
    AND S.course_id = T.course_id
);
```

Question 23

Find courses offered in Fall 2009 but not in Spring 2010.

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
AND course_id NOT IN (
    SELECT course_id
    FROM section
    WHERE semester = 'Spring' AND year = 2010
);
```

Question 24

Find the names of instructors whose names are neither “Mozart” nor “Einstein”.

```
SELECT DISTINCT name
FROM instructor
WHERE name NOT IN ('Mozart', 'Einstein');
```

Question 25

Find the total number of distinct students who have taken course sections taught by the instructor with ID 10101

```
SELECT COUNT(DISTINCT ID)
FROM takes
WHERE (course_id, sec_id, semester, year) IN (
    SELECT course_id, sec_id, semester, year
    FROM teaches
    WHERE teaches.ID = 10101
);
```

```
SELECT COUNT(DISTINCT takes.ID)
FROM takes
JOIN teaches
    ON takes.course_id = teaches.course_id
   AND takes.sec_id = teaches.sec_id
   AND takes.semester = teaches.semester
   AND takes.year = teaches.year
WHERE teaches.ID = 10101;
```

```
SELECT COUNT(DISTINCT takes.ID)
FROM takes
WHERE EXISTS (
    SELECT 1
    FROM teaches
    WHERE teaches.ID = 10101
       AND teaches.course_id = takes.course_id
       AND teaches.sec_id = takes.sec_id
       AND teaches.semester = takes.semester
       AND teaches.year = takes.year
);
```

Question 26

Find the names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
SELECT DISTINCT T.name
FROM instructor AS T, instructor AS S
WHERE T.salary > S.salary
      AND S.dept_name = 'Biology';
```

```
-- Using the > SOME clause
SELECT name
FROM instructor
WHERE salary > SOME (
    SELECT salary
    FROM instructor
    WHERE dept_name = 'Biology'
);
```

Question 27

Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
-- Using the > ALL clause
SELECT name
FROM instructor
WHERE salary > ALL (
    SELECT salary
    FROM instructor
    WHERE dept_name = 'Biology'
);
```

Question 28

Find the departments that have the highest average salary.

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) >= ALL (
    SELECT AVG(salary)
    FROM instructor
    GROUP BY dept_name
);
```

Question 29

Find all students who have taken all courses offered in the Biology department.

```

SELECT DISTINCT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS (
    (SELECT course_id
     FROM course
     WHERE dept_name = 'Biology')
    EXCEPT
    (SELECT T.course_id
     FROM takes AS T
     WHERE S.ID = T.ID)
);

```

Question 30

Find all courses that were offered at most once in 2009.

```

-- Solution 1 (using UNIQUE, not standard SQL)
SELECT T.course_id
FROM course AS T
WHERE UNIQUE (
    SELECT R.course_id
    FROM section AS R
    WHERE T.course_id = R.course_id
    AND R.year = 2009
);

```

```

-- Solution 2 (using COUNT, standard SQL)
SELECT T.course_id
FROM course AS T
WHERE 1 = (
    SELECT COUNT(R.course_id)
    FROM section AS R
    WHERE T.course_id = R.course_id
    AND R.year = 2009
);

```

Question 31

Find all courses that were offered at least twice in 2009.

```
-- Solution 1 (using NOT UNIQUE, non-standard SQL)
SELECT T.course_id
FROM course AS T
WHERE NOT UNIQUE (
    SELECT R.course_id
    FROM section AS R
    WHERE T.course_id = R.course_id
        AND R.year = 2009
);
```

```
-- Solution 2 (using COUNT, standard SQL)
SELECT T.course_id
FROM course AS T
WHERE 2 <= (
    SELECT COUNT(R.course_id)
    FROM section AS R
    WHERE T.course_id = R.course_id
        AND R.year = 2009
);
```

Question 32

Find the average instructors' salaries of those departments where the average salary is greater than \$42,000.

```
SELECT dept_name, avg_salary
FROM (
    SELECT dept_name, AVG(salary) AS avg_salary
    FROM instructor
    GROUP BY dept_name
) AS dept_avg
WHERE avg_salary > 42000;
```

Question 33

Find the names of each instructor, along with their salary and the average salary in their department.

```
SELECT name, salary, avg_salary
FROM instructor I1,
LATERAL (
    SELECT AVG(salary) AS avg_salary
    FROM instructor I2
    WHERE I2.dept_name = I1.dept_name
) AS dept_avg;
```

Question 34

Find all departments with the maximum budget.

```
-- Using a CTE or subquery to get the maximum budget
WITH max_budget (value) AS (
    SELECT MAX(budget)
    FROM department
)
SELECT dept_name, budget
FROM department, max_budget
WHERE department.budget = max_budget.value;
```

Question 35

Find all departments where the total salary is greater than the average of the total salary across all departments.

```
-- Using CTEs to calculate total salaries and their average
WITH dept_total (dept_name, value) AS (
    SELECT dept_name, SUM(salary)
    FROM instructor
    GROUP BY dept_name
),
dept_total_avg (value) AS (
    SELECT AVG(value)
    FROM dept_total
)
SELECT dept_name
FROM dept_total, dept_total_avg
WHERE dept_total.value >= dept_total_avg.value;
```

Question 36

List all departments along with the number of instructors in each department.

```
SELECT dept_name,
    (SELECT COUNT(*)
     FROM instructor
     WHERE department.dept_name = instructor.dept_name) AS num_instructors
FROM department;
```

Question 37

Delete all instructors from the instructor table.

```
DELETE FROM instructor;
```

Question 38

Delete all instructors from the Finance department.

```
DELETE FROM instructor
WHERE dept_name = 'Finance';
```

Question 39

Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.

```
DELETE FROM instructor
WHERE dept_name IN (
    SELECT dept_name
    FROM department
    WHERE building = 'Watson'
);
```

Question 40

Delete all instructors with a salary between \$13,000 and \$15,000.

```
DELETE FROM instructor
WHERE salary BETWEEN 13000 AND 15000;
```

Question 41

Delete all instructors whose salary is less than the average salary of instructors.

```
DELETE FROM instructor
WHERE salary < (
    SELECT AVG(salary)
    FROM instructor
);
```

Question 42

Add a new tuple to the course relation.

```
-- Option 1: Insert all values in order
INSERT INTO course
VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

-- Option 2: Specify columns explicitly
INSERT INTO course (course_id, title, dept_name, credits)
VALUES ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

Question 43

Add a new tuple to the student relation with total creds set to NULL.

```
INSERT INTO student
VALUES ('3003', 'Green', 'Finance', NULL);
```

Question 44

Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department, with a salary of \$18,000.

```
INSERT INTO instructor
SELECT ID, name, dept_name, 18000
FROM student
WHERE dept_name = 'Music' AND tot_cred > 144;
```

Question 45

Increase salaries of all instructors by 5 percent.

```
UPDATE instructor
SET salary = salary * 1.05;
```

Question 46

Increase the salary of instructors by 7 percent if their salary is less than \$70,000.

```
UPDATE instructor
SET salary = salary * 1.07
WHERE salary < 70000;
```

Question 47

Give a 5 percent salary raise to instructors whose salary is less than the average salary.

```
UPDATE instructor
SET salary = salary * 1.05
WHERE salary < (
    SELECT AVG(salary)
    FROM instructor
);
```

Question 48

Increase salaries of instructors whose salary is over \$100,000 by 3 percent and all others receive a 5 percent raise.

```
-- Raise 3% for high-paid instructors
UPDATE instructor
SET salary = salary * 1.03
WHERE salary > 100000;
```

```
-- Raise 5% for all other instructors
UPDATE instructor
SET salary = salary * 1.05
WHERE salary <= 100000;
```

```
update instructor
set salary = case
when salary <= 100000 then salary * 1.05
else salary * 1.03
end
```

Question 49

Update the total credits (total cred) of each student. A course is considered successfully completed if the student has a grade that is not F or NULL. Set total cred to 0 for students who have not successfully completed any course.

```

-- Update tot_cred using sum of credits (NULL if no courses)
UPDATE student S
SET tot_cred = (
    SELECT SUM(credits)
    FROM takes NATURAL JOIN course
    WHERE S.ID = takes.ID
        AND takes.grade <> 'F'
        AND takes.grade IS NOT NULL
);

-- Update tot_cred replacing NULL with 0
UPDATE student S
SET tot_cred = (
    SELECT CASE
        WHEN SUM(credits) IS NOT NULL THEN SUM(credits)
        ELSE 0
    END
    FROM takes NATURAL JOIN course
    WHERE S.ID = takes.ID
        AND takes.grade <> 'F'
        AND takes.grade IS NOT NULL
);

```

Question 50

Create a table with the same schema as an existing table.

```

-- Option 1: Create a table with the same schema (no data)
CREATE TABLE temp_account LIKE account;

-- Option 2: Create a table with the same schema and copy all data
CREATE TABLE temp_account AS
SELECT *
FROM account;

```